ENTERPRISE TECHNICAL SPECIFICATION & VIVA GUIDE

Service Center Management System (SCMS)

Comprehensive Technical Architecture, Business Logic & Operational Lifecycle Documentation

Domain	Reference Prototype	Architecture	Security & Tax
Electronics After-Sales Care	Samsung Authorized Care Model	Decoupled 3-Tier MVC + SPA	RBAC • JWT • 18% GST Compliant

Document Navigation Index

01	Executive Summary & Industry Background	06	Security Architecture & RBAC Matrix
02	Problem Statement & Engineered Solutions	07	Step-by-Step Scenario Walkthroughs
03	System Architecture & Full-Stack Technologies	08	Comprehensive API Architecture Directory
04	Relational Database Design & Schema Entities	09	Technical Viva & Examiner Q&A Guide
05	Core Business Logic, State Engines & Formulas	10	Project Evaluation & Portfolio Takeaways

1. EXECUTIVE SUMMARY & BACKGROUND

The **Service Center Management System (SCMS)** is an enterprise web application architected to modernize, automate, and digitize the end-to-end operations of high-volume electronics after-sales service centers. The operational design, business constraints, state machines, and billing models implemented in SCMS were inspired by standard operating procedures evaluated at an authorized **Samsung Customer Care Service Center**.

In authorized service facilities, daily performance hinges upon rapid turnaround time (TAT), inventory accountability, zero unbilled component shrinkage, and clear synchronization between front-desk service receptionists, technical bench engineers, and walk-in consumers. SCMS replaces fragmented legacy paper job cards, disconnected spreadsheets, and verbal status queries with a secure, auditable, multi-role web platform.

Core Mission & Scope

Tracking customer hardware through its complete lifecycle: customer intake, deterministic warranty verification, technician assignment, diagnostic isolation, spare parts inventory allocation with atomic concurrency locks, secondary quality testing, GST-compliant financial invoicing, and formal signed device handover.

2. PROBLEM STATEMENT & REAL-WORLD MOTIVATION

Traditional electronics service centers face recurring operational friction when utilizing manual methods. The table below outlines these industry bottlenecks alongside SCMS's engineered technological countermeasures:

Operational Challenge	Real-World Impact & Friction	SCMS Engineered Solution
Manual Job Registers	Physical job sheets are vulnerable to loss, water damage, illegible technician logs, and excruciatingly slow customer retrieval.	Searchable relational database indexing customer phone numbers, ticket codes, and device serial numbers with indexed sub-millisecond retrieval.
Lack of Real-Time Tracking	Customers repeatedly call or visit the front desk for updates, pulling receptionists away from onboarding new customers.	Deterministic 7-stage state machine paired with automated transaction timelines and email notification triggers on status transitions.
Untracked Spare Parts	Stockouts midway through bench repair, component leakage/shrinkage, and unbilled parts handed back inside devices.	Atomic transactional stock decrement upon ticket attachment, real-time snapshot billing capture, and automated low-stock reorder thresholds.
Warranty Ambiguity	Manual receipt review causes cashier disputes, human errors, and fraudulent duplicate warranty claims on repaired items.	Automated mathematical rule engine validating purchase date and device tier against global unique hardware serial constraints.
Manual Billing Errors	Inconsistent labor rates, unstandardized tax applications, and lack of printed compliant invoices lead to tax audit vulnerabilities.	Automated billing engine computing ***Parts + Labor + 18% GST - Discount*** with browser-printable Samsung Care receipts.
Unbalanced Tech Loading	Senior technicians become overloaded while newer technicians remain idle, degrading overall center turnaround SLA.	Dynamic technician workload queue ranking engineers by count of active, non-delivered tickets during ticket dispatch.

3. SYSTEM ARCHITECTURE & TECHNOLOGY STACK

SCMS utilizes a decoupled three-tier architecture built on the **Model-View-Controller (MVC)** paradigm on the backend and a reactive single-page architecture (SPA) on the client tier.

TIER 1: PRESENTATION TIER (REACT 18 + VITE + TAILWIND CSS)

• **React 18:** Component-driven reactive UI employing functional hooks (`useState`, `useEffect`, `useContext`) for modular state management.
• **Vite:** Ultra-fast developer tooling leveraging native ES modules (ESM) and pre-bundling with `esbuild`, replacing deprecated Create-React-App.
• **Tailwind CSS:** Styled to match the official Samsung corporate palette: Samsung Blue (`#1428A0`), Deep Navy (`#0D1B6F`), Action Blue (`#4285F4`), and Clean Tint (`#F8F9FA`).
• **Recharts:** Interactive SVG charting for operational dashboards including intake time series, repair status distributions, and revenue graphs.
• **Axios Interceptors:** Global request interceptors injecting JWT Bearer tokens and response interceptors handling automatic 401 unauthenticated session expiry.

TIER 2: APPLICATION / LOGIC TIER (NODE.JS + EXPRESS.JS REST API)

• **Express.js MVC:** Strict separation of concerns across route definitions, controllers, and business service layers.
• **Input Sanitization:** Strict payload validation with `express-validator` guarding all create/update endpoints before database calls.
• **Hardened Security:** HTTP protection headers configured with `helmet` (HSTS, CSP, XSS-Filter, Frameguard) and whitelist-only CORS.

TIER 3: PERSISTENCE TIER (SEQUELIZE ORM + DUAL DIALECT ENGINE)

• **Sequelize ORM:** Centralized object-relational mapping enforcing strict associations, paranoid soft-deletes, and parameterized query generation.
• **Dual Engine Adaptability:** Runs seamlessly on `SQLite3` for zero-configuration viva evaluations, local development, and CI/CD testing; switches to `PostgreSQL` with connection pooling for production deployments via environment flags.

4. DATABASE DESIGN & ENTITY RELATIONSHIPS

The persistence model is architected around 9 normalized tables enforcing strict foreign key constraints and referential integrity:

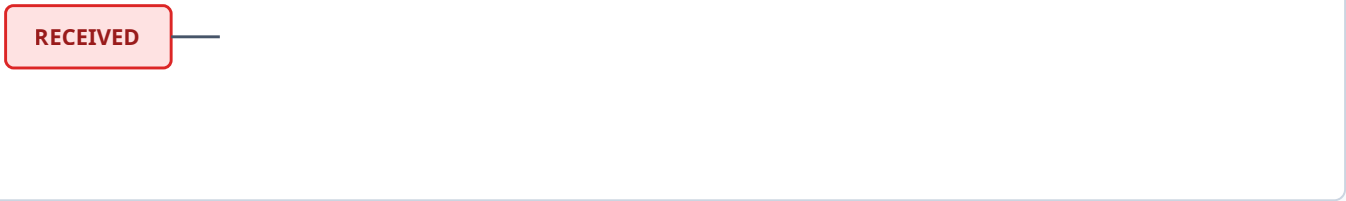
Relational Schema Architecture (ERD Summary)

Entity Model	Primary Key	Foreign Keys & Constraints	Core Attributes & Types
USERS	`id (INT)`	`email` (UNIQUE)	`name`, `password_hash`, `role (admin, receptionist, technici`
CUSTOMERS	`id (INT)`	`phone` (UNIQUE)	`name`, `email`, `address`, `is_active`
DEVICES	`id (INT)`	`customer_id` (FK), `serial_number` (UK)	`brand`, `model`, `category`, `purchase_date`, `warranty_period_`
SERVICE_TICKETS	`id (INT)`	`ticket_number` (UK), `customer_id` (FK), `device_id` (FK), `technician_id` (FK)	`status (ENUM)`, `priority (ENUM)`, `complaint_description`, `c` `completed_at`, `delivered_at`
SPARE_PARTS	`id (INT)`	`part_number` (UNIQUE)	`part_name`, `category`, `quantity_in_stock`, `unit_price`, `reo`
TICKET_PARTS	`id (INT)`	`ticket_id` (FK), `part_id` (FK)	`quantity_used`, `price_charged (DECIMAL snapshot)`
INVOICES	`id (INT)`	`invoice_number` (UK), `ticket_id` (FK), `customer_id` (FK)	`parts_cost`, `labor_cost`, `subtotal`, `tax_rate (18%)`, `tax_ar` `payment_status`, `payment_method`, `paid_at`
TICKET_TIMELINES	`id (INT)`	`ticket_id` (FK), `changed_by` (FK)	`status`, `notes`, `createdAt (TIMESTAMP)`
NOTIFICATIONS	`id (INT)`	`ticket_id` (FK), `customer_id` (FK)	`recipient`, `subject`, `channel (EMAIL)`, `status (SENT, FAILE`

5. CORE BUSINESS LOGIC & STATE ENGINES

5.1 Repair Lifecycle State Machine

Service tickets follow an explicit, single-direction operational lifecycle. Every transition automatically records an immutable timeline snapshot:



RECEIVED RED **:** Device physical intake. Sequential tracking code (`SVC-YYYYMMDD-NNN`) generated. Intake defects and symptoms recorded.

DIAGNOSING ORANGE **:** Technician disassembles device; electrical component testing and microscopic analysis logged.

WAITING_FOR_PARTS YELLOW **:** Required OEM component is out-of-stock locally; purchase request flagged to regional warehouse.

REPAIRING BLUE **:** Active workbench replacement, micro-soldering, adhesive curing, and reassembly.

READY GREEN **:** Hardware passes quality diagnostics; `completed_at` timestamp saved; pickup email dispatched.

DELIVERED SLATE **:** Customer settles invoice balance, tests device at counter, and signs handover receipt.

CANCELLED DARK RED **:** Device deemed irreparable or customer declines repair estimate.

5.2 Dynamic Warranty Computation Engine

When a customer hardware item is registered, SCMS calculates warranty validity:

$$\textit{Warranty Expiration Date} = \textit{Purchase Date} + (\textit{Warranty Period in Months})$$

Status is evaluated dynamically against the system clock:

$$\textit{Current Date} \leq \textit{Warranty Expiry} \;\&implies;\; \textit{Under Warranty} \quad | \quad \textit{Current Date} > \textit{Warranty Expiry} \;\&implies;\; \textit{Out of Warranty}$$

Fraud Protection: Hardware serial numbers are strictly globally unique, preventing multiple customer claims against identical components.

5.3 Inventory Consumption & Concurrency Safeguards

To eliminate race conditions in high-volume environments, spare parts management runs inside isolated database transactions:

Transactional Stock Deduction: When a part is attached to a ticket, a Sequelize transaction starts. If `quantity_in_stock < requested`, the transaction rolls back immediately with `400 Insufficient Inventory`. If sufficient, the stock is decremented, snapshot unit pricing is written to `ticket_parts`, and the transaction commits.

Stock Restoration: If a technician unlinks a part, the allocated quantity is credited back to `spare_parts.quantity_in_stock` atomically.

3. **Threshold Alerts:** When $Stock \leq Reorder\ Level$, items are flagged with a low-stock alert on the administrative console.

5.4 Billing & GST Computation Formula

$$Parts\ Cost = \Sigma\ (Quantity\ Used \times Unit\ Snapshot\ Price)$$
$$Subtotal = Parts\ Cost + Labor\ Charges$$
$$GST\ Amount\ (18\%) = Subtotal \times 0.18$$
$$Total\ Payable = max(0, Subtotal + GST\ Amount - Discount)$$

Invoices generate standardized daily sequential identifiers (`INV-YYYYMMDD-NNN`). Upon payment settlement via UPI, Card, or Cash, transaction authorization codes and `paid_at` timestamps are recorded, unlocking browser-printable receipts.

5.5 Workload Balancing & SLA Tracking

$$Active\ Workload = Count(Tickets\ in\ \{RECEIVED, DIAGNOSING, WAITING_FOR_PARTS, REPAIRING\})$$

Engineers with workload < 5 are marked **Available**; those with ≥ 5 are marked **High Load**. Turnaround SLA performance is computed across all resolved tickets:

$$Average\ Turnaround\ Hours = \Sigma\ (completed_at - createdAt) / Total\ Resolved\ Jobs$$

6. SECURITY ARCHITECTURE & RBAC IMPLEMENTATION

SCMS implements defense-in-depth across authentication, transport, data models, and authorization guards:

- **Stateless JWT:** Authentication tokens signed with HMAC-SHA256 containing user ID and role, expiring after 24 hours.
- **Bcrypt Hashing:** Passwords salted and hashed with 10 salt rounds before storage; never written to plaintext.
- **Sanitized Serializers:** User entity definitions implement custom `toJSON()` logic stripping `password_hash` from all API payloads.

Role-Based Access Control (RBAC) Matrix

Endpoint / Operation	Method	Admin	Receptionist	Technician
`/api/auth/register` (Create Staff)	POST	ALLOW	DENY	DENY
`/api/customers` (Customer Directory)	GET / POST	ALLOW	ALLOW	DENY
`/api/tickets` (Create Ticket)	POST	ALLOW	ALLOW	DENY
`/api/tickets/:id/status` (Update Status)	PATCH	ALLOW	ALLOW	Assigned Only
`/api/tickets/:id/parts` (Consume Spare Part)	POST	ALLOW	DENY	Assigned Only
`/api/inventory` (Manage & Restock Parts)	POST / PATCH	ALLOW	DENY	DENY
`/api/billing/generate` (Create Invoice)	POST	ALLOW	ALLOW	DENY
`/api/reports/*` (Analytics & KPIs)	GET	ALLOW	DENY	DENY

7. STEP-BY-STEP SCENARIO WALKTHROUGHS

Scenario 1: Customer Walk-in & Intake (Receptionist)
Customer presents a damaged Samsung Galaxy S24 Ultra. The receptionist enters customer contact information and registers the device model and serial (`SN-S24U-9901`). The warranty engine verifies purchase date validity. The receptionist inputs symptom notes ("Screen fractured, digitizer dead"), queries `/api/technicians/available`, selects the engineer with the lowest active load, and creates ticket `SVC-20260903-001`. An automated intake confirmation email is dispatched.

Scenario 2: Diagnostic Inspection & Notes Logging (Technician)
The technician filters the workbench view to "My Assigned Tickets", opens `SVC-20260903-001`, and clicks **Start Diagnosis** (status shifts to `DIAGNOSING`, triggering a customer email). After disassembly and microscope analysis, the technician logs findings: *"AMOLED panel shattered, ribbon cable torn, motherboard and battery intact"* and saves diagnostic notes.

Scenario 3: Parts Attachment & Repair Bench (Technician)
The technician confirms stock for a *Dynamic AMOLED 2X Panel* (stock: 6 units). Clicking **Add Part** requests 1 unit. An atomic database transaction decrements stock from 6 to 5 and binds the snapshot price (₹18,500) to the ticket. The technician clicks **Start Repair** (status → `REPAIRING`), fits the screen, reapplies water-resistant adhesive, and runs screen calibration tests.

Scenario 4: Secondary Quality Check & Ready Notification (Technician)
The hardware passes full diagnostic routines (touch grid, ambient sensor, camera, speakers). The technician clicks **Mark Ready**. The system updates the status to `READY`, writes the `completed_at` timestamp for SLA tracking, and emails the customer that their device is ready for collection.

Scenario 5: Settlement, Handover & Official Invoicing (Receptionist)
The customer arrives at the counter. The receptionist clicks **Generate Invoice**. The billing engine pulls the ₹18,500 parts cost and applies a standard ₹500 labor fee: Subtotal = ₹19,000; 18% GST = ₹3,420; Total = ₹22,420. The customer completes UPI payment. The receptionist records the UPI authorization code, changes payment status to `PAID`, updates ticket status to `DELIVERED` (logging `delivered_at`), and prints the official Samsung Care receipt.

8. API ARCHITECTURE & ENDPOINTS DIRECTORY

All endpoints are structured under `/api` and return JSON responses formatted with standard status envelopes:

Module	Method	Endpoint	Functional Responsibility & Query Params
Auth	POST	`/api/auth/login`	Verifies email and password; generates HMAC-SHA256 JWT bearer token.
	GET	`/api/auth/me`	Returns active authenticated user context and granted roles.
	POST	`/api/auth/register`	Admin endpoint registering new staff accounts with bcrypt password salting.
Customers	GET	`/api/customers`	Searchable, paginated directory with attached device counts.
	POST	`/api/customers`	Registers customer profile with unique phone number constraint check.
Tickets	GET	`/api/tickets`	Queries tickets with filters (`status` , `priority` , `technicianId` , `search`).
	POST	`/api/tickets`	Generates job card with standardized code (`SVC-YYYYMMDD-NNN`).
	PATCH	`/api/tickets/:id/status`	Transitions state engine; writes immutable audit record to `ticket_timelines` .
	POST	`/api/tickets/:id/parts`	Consumes replacement component within atomic ACID database transaction.
Inventory	GET	`/api/inventory/low-stock`	Fetches parts where `quantity_in_stock ≤ reorder_level` for replenishment.
	PATCH	`/api/inventory/:id/restock`	Increments stock quantities upon receiving physical warehouse shipments.
Billing	POST	`/api/billing/generate`	Builds official invoice (`INV-YYYYMMDD-NNN`) applying 18% GST calculations.
	PATCH	`/api/billing/:id/pay`	Settles payment balance (UPI, CARD, CASH), stamping `paid_at` .

9. PROJECT VIVA & TECHNICAL INTERVIEW Q&A GUIDE

Q1: Why did you choose Sequelize ORM instead of writing raw SQL queries?

Sequelize provides relational model definitions, built-in validation, and relationship abstractions (1:N, N:M). It abstracts database dialects so the application runs locally on zero-setup SQLite3 during demonstrations and switches to PostgreSQL in production through simple environment variables. Sequelize also automatically parameterizes queries, protecting against SQL Injection attacks.

Q2: How is data consistency maintained when a technician consumes a spare part?

The operation is wrapped inside a managed ACID transaction using `sequelize.transaction()`. Inside the boundary, the engine verifies that `quantity_in_stock ≥ requested_quantity`. It decrements stock in `SparePart`, logs a junction record in `TicketPart` with the snapshot price, and creates an audit entry in `TicketTimeline`. If any step throws an error, the entire transaction rolls back, preventing stock discrepancies.

Q3: How does the system handle sequential ticket and invoice numbering?

Exposing raw database auto-increment IDs leaks business volume to competitors. Instead, a dedicated service (`src/services/numberGenerator.js`) generates formatted codes like `SVC-YYYYMMDD-NNN` and `INV-YYYYMMDD-NNN`. It queries the database for the highest sequence number stamped on the current calendar date, increments it by 1, and pads the value to three digits, matching enterprise ERP conventions.

Q4: How does dynamic warranty calculation function, and what edge cases were addressed?

Upon device intake, the user provides the purchase date and warranty tenure in months. The database calculates the expiration date (`purchase_date + months`). A virtual attribute compares this against the system date: if `Current Date ≤ Expiry`, it flags the ticket as Under Warranty. Hardware serial numbers enforce unique constraints globally, preventing duplicate claims on identical devices.

Q5: Why select Vite over traditional Create-React-App (CRA)?

Create-React-App is deprecated and relies on Webpack, which rebuilds entire dependency bundles on code modifications. Vite leverages native ES Modules (ESM) and pre-compilation via `esbuild` written in Go, providing near-instant server boot times and sub-millisecond Hot Module Replacement (HMR), alongside optimized tree-shaken production bundles via Rollup.

Q6: How does the system prevent unbilled tickets from being released to customers?

The delivery action is gated at both client and API controller levels. The frontend only enables the "Deliver to Customer" action once the ticket has an attached invoice marked with payment status `PAID`. The backend controller validates that non-warranty tickets possess a paid settlement record before allowing a transition to `DELIVERED`.

Q7: How did you implement defensive API security?

Security in SCMS follows defense-in-depth: Helmet sets HTTP security headers (HSTS, Content Security Policy, X-Frame-Options); CORS whitelists the frontend domain; `express-validator` sanitizes and validates incoming payloads; passwords are salted and hashed with `bcryptjs`; JWTs expire in 24 hours; and user JSON serializers strip password hashes from all outbound payloads.

Q8: What design system principles did you apply to the user interface?

The interface follows the official Samsung Corporate Visual Identity. It features Samsung Blue (`#1428A0`) as the primary brand color, dark slate typography (`#0F172A`), WCAG-compliant status badge contrasts, consistent 150ms transitions on interactive elements, and standardized SVG icons from `lucide-react`, omitting informal emojis in favor of an enterprise aesthetic.

10. PROJECT EVALUATION & PORTFOLIO TAKEAWAYS

The **Service Center Management System (SCMS)** is an enterprise operational platform designed to demonstrate key engineering competencies:

- **Solves Concrete Real-World Problems:** Addresses operational workflows modeled directly from authorized Samsung care facilities.
- **Technical Rigor:** Features deterministic state engines, transactional concurrency locks, mathematical GST billing, and automated SLA tracking.
- **Code Quality:** Decoupled MVC modularity, test-driven validation (33 automated integration tests), and zero-configuration SQLite/PostgreSQL flexibility.
- **Professional Usability:** Enterprise UI/UX with responsive layouts, accessible typography, and browser-printable invoices.